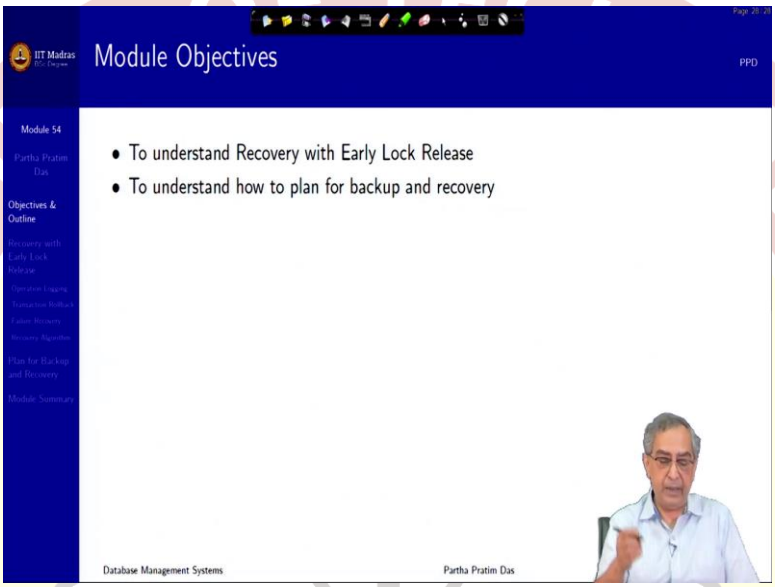
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Backup & Recovery 4: Recovery 3

Welcome to Module 54 of Database Management Systems in the IIT Madras online BSc program.

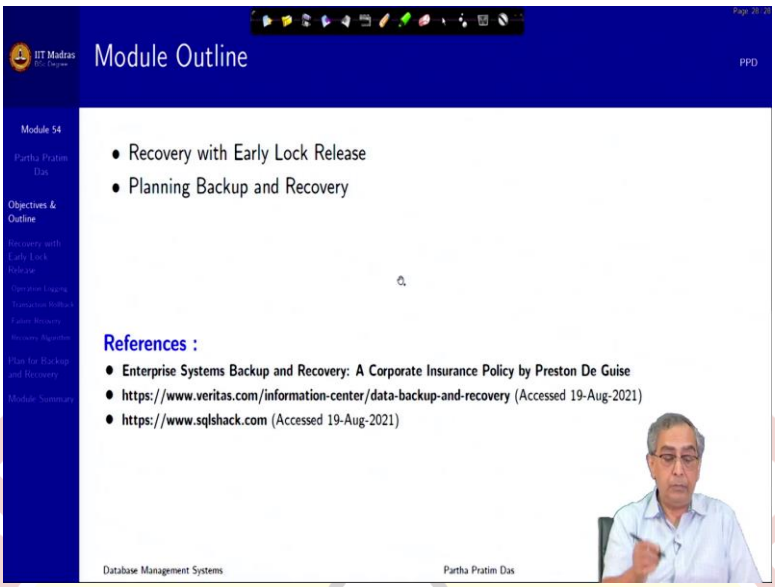
(Refer Slide Time: 0:26)

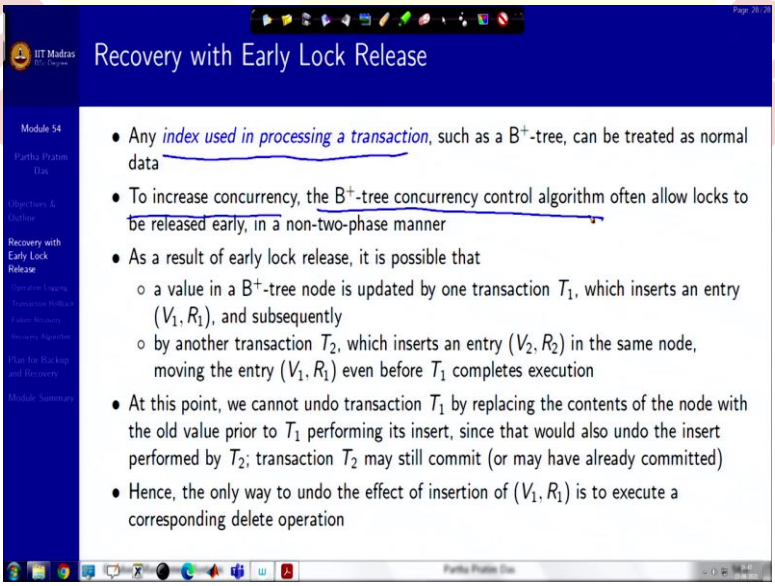


We have been discussing about various nuances of backup and recovery in a database. In the last module, we saw how hot backup of transaction log can help recovering to a consistent database and then based on that we understood and so an example of recovery algorithm for concurrent transactions.

In this, we will take this issue of recovery to some final realistic levels I should say that we have always been so far looking at the recovery at the transaction level only and thinking that okay read, write has been done and that has to be undone and things like that. But in reality, when you look into what is going to happen on the desk, what is going to happen in terms of your actual data file, index files and so on, you will see that there are more nuances involved in the recovery. So, let us look into that part. And then we briefly present a possible plan for backup and recovery not exactly a plan, but kind of a walk through the parameters on which an organization will decides, decide its backup and recovery plan.

(Refer Slide Time: 1:49)





So, we discuss about recovery with early lock release. First, let us understand the situation. We are saying that we have done a write, after that write the buffer manager at some point, will write that block from the memory to the database. Now, when it writes to the database that is on the on the B plus tree certainly, there is going to be some issues again of concurrency, because multiple buffers may be written with different value updates at the same point of time.

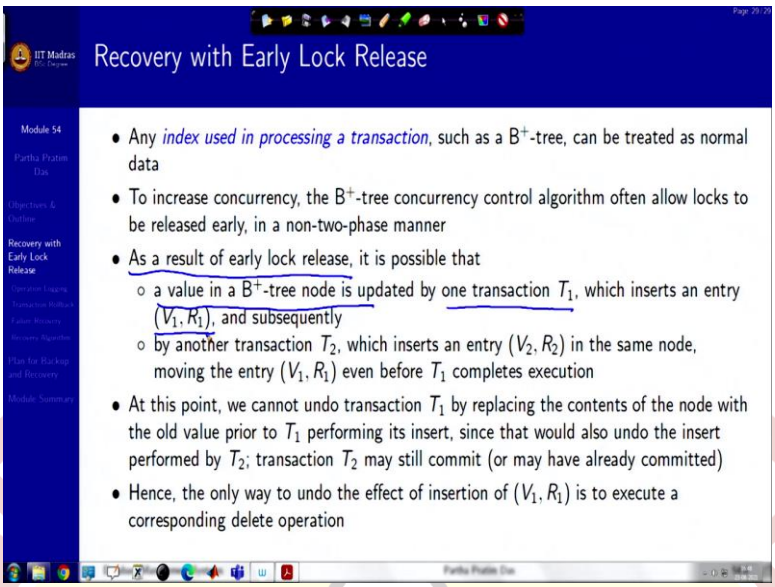
So, again, as you do for at a transaction level, even at this low level of updating the files, you will have to use some low-level locking mechanism, we are not going into the that kind of locking mechanism, but some kind of locking mechanism will have to be there. So, let us say, let us consider that we are talking about the index that is used in processing a transaction. We know indexes will have to be there, so that we can do fast access of the data.

Now, naturally to increase the concurrency the B plus tree concurrency algorithm which allows concurrent write, concurrent read and all those shared exclusive stuff with the B plus tree for the index for the data and so on. Now, the often that algorithm often has to allow locks to be released early in a non-two-phase manner. Why?

Because if it does not since multiple transaction, this is a centralized resource. The B plus trees are centralized resource with multiple transactions requiring them. So, if you hold the lock for the commit of one particular transaction by doing a two phasing because two phase as you know, you only acquire in one phase and you release in the other phase and that means that your release can happen only at the time of commit, then the overall throughput is going to become very slow.

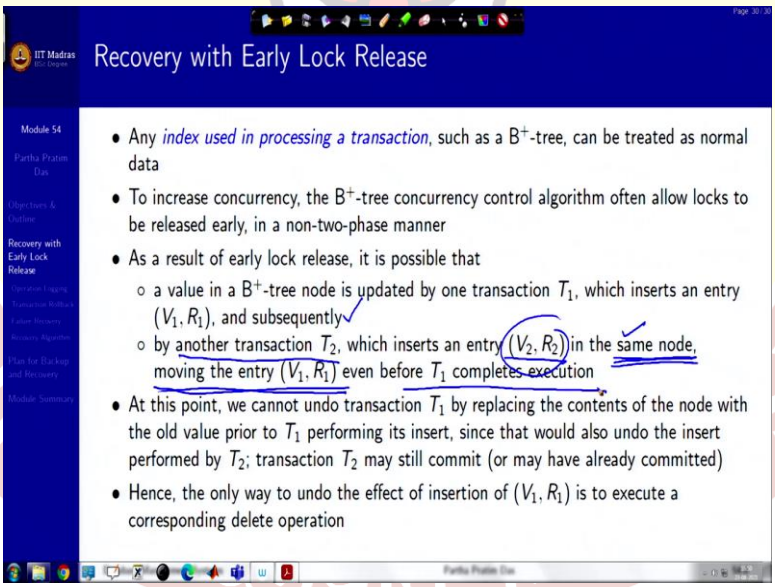
So, you use lock only for the purpose of writing the data on the tree, but do not want to relate it to the lifetime of the transaction. So, if you make that, if you do that, that is what the B plus concurrency control algorithms do. There is a direct effect. So, as a result of this, this is called early lock release early in terms of early with respect to that right transaction having been completed.

(Refer Slide Time: 5:04)



Now, what is possible that a value in B plus tree node is updated by one transaction T1 which enters an entry V1 R1. We are just taking the case of index B trees, B plus trees. So, it is putting inserting an entry value 1 reference 1.

(Refer Slide Time: 5:28)



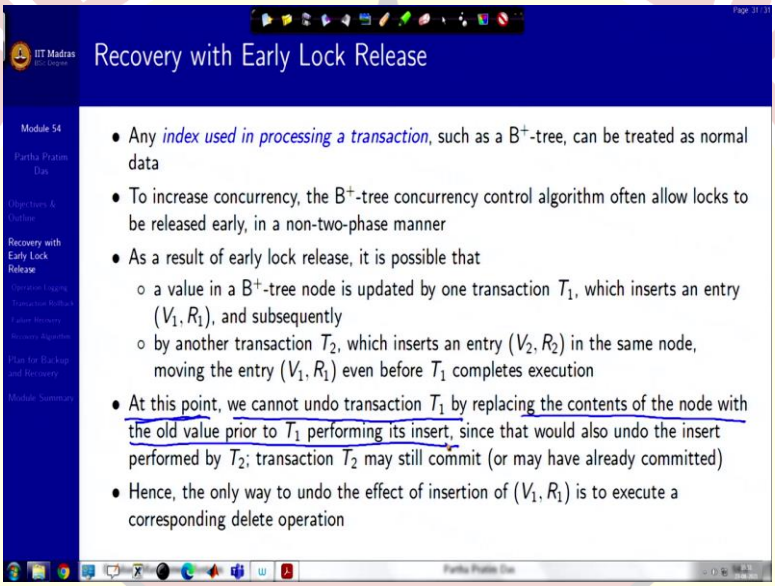
Now, at that time naturally you needed a low-level lock, but due to early release that is release, so another transaction T2 inserts V2 R2 in the same node, you can enter insert in the same node because you have released the lock early. Otherwise, when you updated this one you obviously held a low-level lock on that node, so that your write does not get disturbed.

But after that you have left it and T1 has not completed, before the completion T2 is writing another entry in the same node. And the moment you write that think about the B plus tree

structure, the writes are not independent, because the moment you write it, there may be some movements of the nodes, there may be some node splits happening, nodes moving up down, records moving inside the node and so on.

So, the entry V1 R1 will move even before T1 completes the transaction completes execution. This part should be clear that one part is T1 has written that data into the B plus tree, V1 R1. But due to some other transaction being able to write to that another entry, insert another entry in the same node before T1 completes execution, the entry position of V1 R1 has changed even before T1 has completed execution.

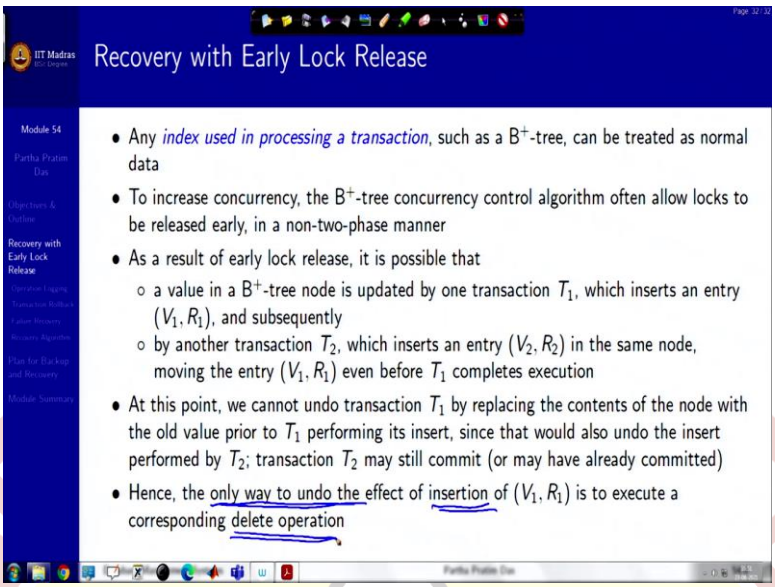
(Refer Slide Time: 7:12)



And at this point, if something is required, we cannot undo the transaction T1 by doing a compensation log kind of stuff, by replacing the content of the node with the old value prior to T1, that is a basic compensation idea, you keep the old value, keep the new value, if you have to undo, change the new value to the old value.

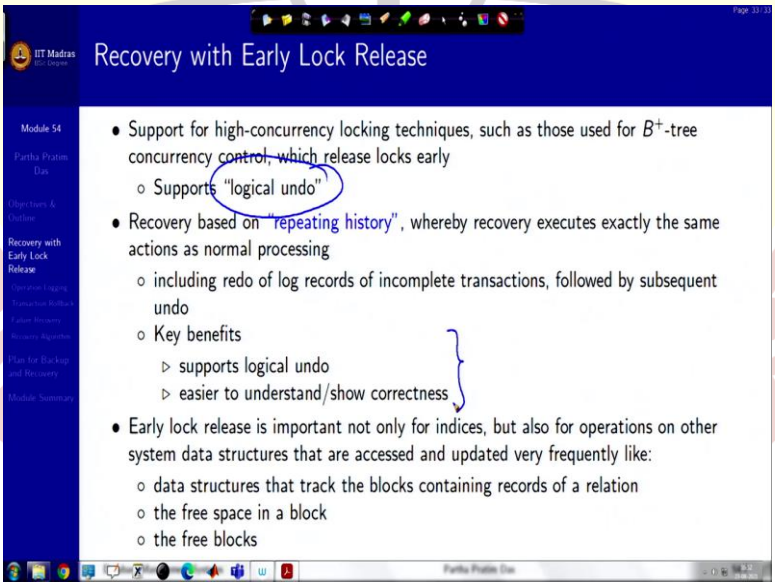
Now, if you do that, in this case, then you lose you also undo the insert performed by T2. So, and T2 may have committed or may still have to commit. So, you wanted to undo V1 R1, but since you will have to write the node, the old value of the node, he will actually undo V1 R1 as well as V2 R2 which is for a different transaction. So, in order to bring the database back to a consistent state, because you are rolling back to 1, you actually create a new inconsistent state.

(Refer Slide Time: 8:27)



So, the only way, the only way to undo the effect of insertion is simply to execute a deletion. You do not need the same, you do not mean, neither you need the same value of the node with all its other records, all that you need is logically V1 R1 which was inserted must not be there in the tree and how do you get there is by delete operation.

(Refer Slide Time: 9:03)

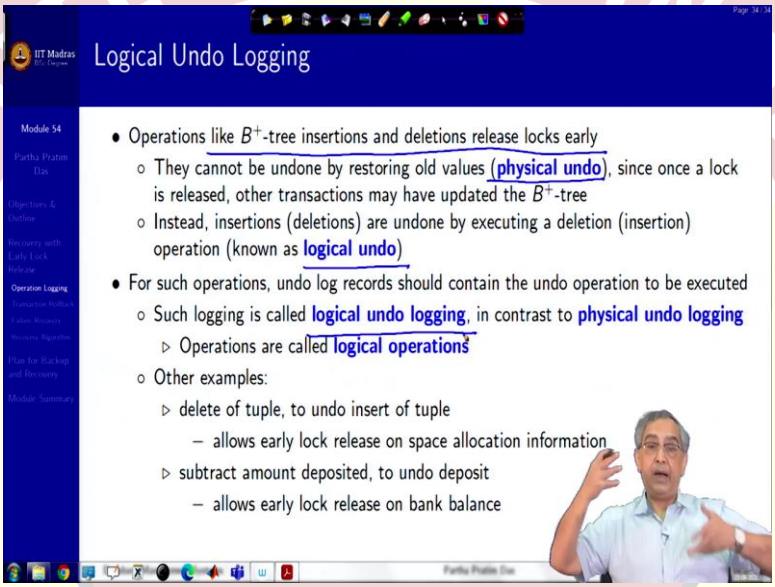


So, this is the core understanding of the issue of recovery with early lock release. So, which, for which you do a logical undo the recovery that we have done so far is has been repeating history whereby recovery executes exactly the same action as normal processing including redo of log records of incomplete transactions followed by subsequent undo. But now he will

have to in this support logical undo and once you do that, it becomes much easier to understand the whole process show its correctness and so on.

And remember early lock release is not only for indices, but even for any other system data structure that are accessed and updated frequently as a consequence of things that the transactions do; for example, data structures that track the blocks containing records of a relation, data structure that tracks the free space of a block, the free blocks themselves and so on.

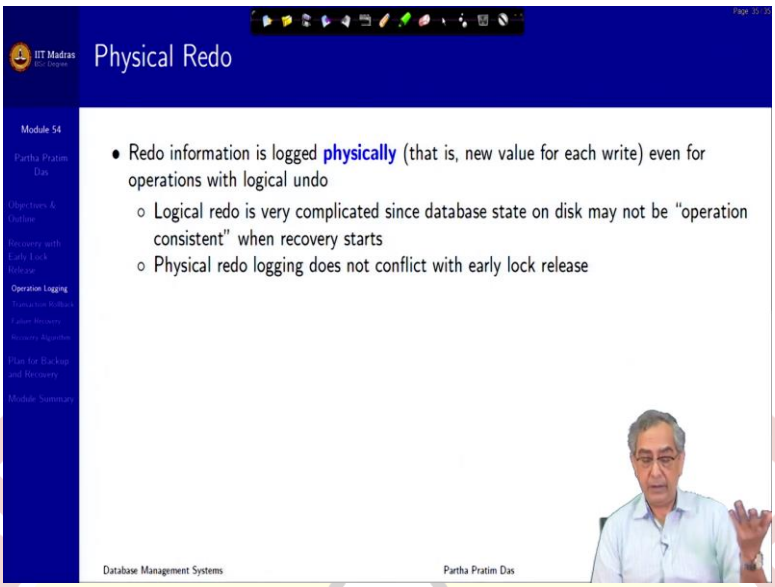
(Refer Slide Time: 10:22)



So, early lock release is it is a very required situation and for that, you use this logical undo. So, what you do is operations like a B plus insertion and deletion release locks early. So, they will not be restored by physical undo instead they will be handled by logical undo; insertion will be undone by deletion, deletion will be undone by insertion.

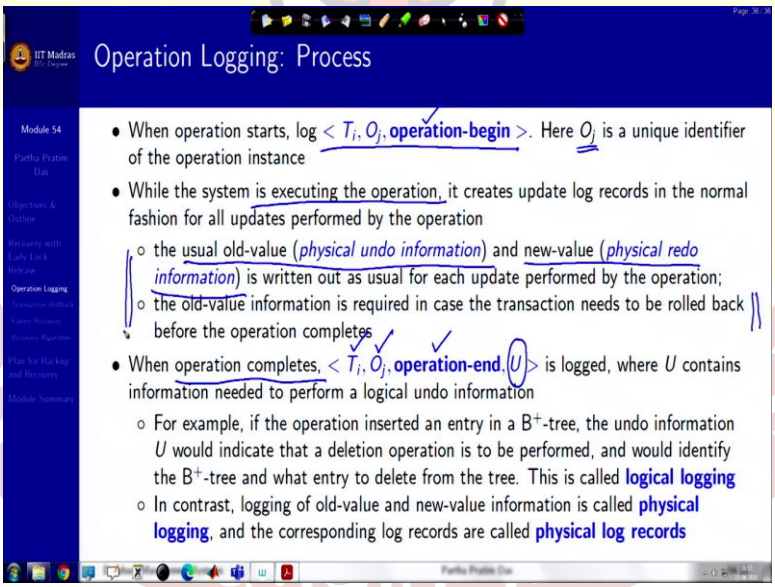
And such logging, the logging for these kinds of requirements is called logical undo logging in contrast to what we will now call the earlier one as physical undo logging. And these operations are called logical operations and more to give it a name it is a logical operation because in contrast to what you were doing, which was physical making the actual changes here you are remembering what kind of counter operation can be done to logically annul the effect that the earlier operation had created, which you are desiring to undo.

(Refer Slide Time: 11:35)



So, redo will be physical they will be locked physically, logical redo becomes very complicated. So, what you do is physical redo logging does not conflict with early lock release, because you can always along with that you can always do a logical undo if required.

(Refer Slide Time: 12:05)

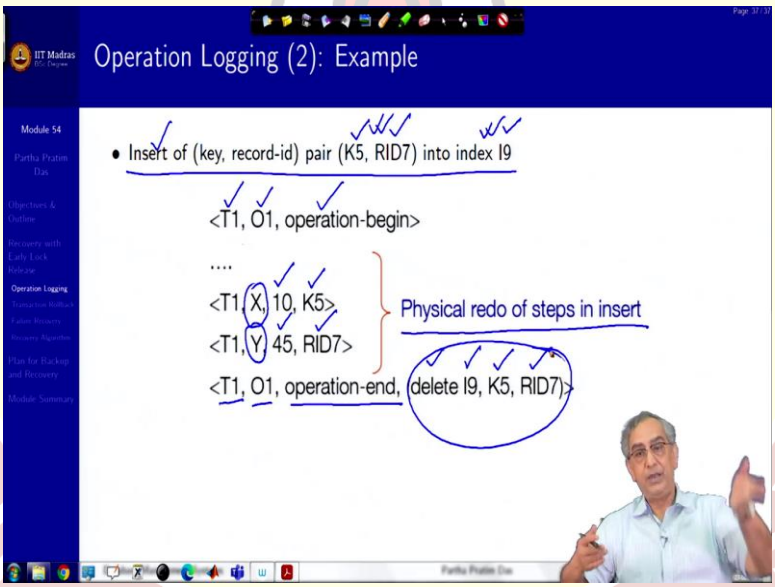


But the operation logging process how will it now look like? So, now we are talking about logging in the case of being doing things logically and taking care of the early lock release. So, what do you log now is an operation. So, you say transaction Ti is causing an operation Oj, well Oj is a unique identifier of the operation, it may be an insert operation and then you say this operation begin so that is where the logging for this operation starts, so the operation has started.

Now, when the system executes operation, it will create update lock records this value is written, this value is written and so on in the usual fashion, that is usual old value which is physical undo information and the new value which is physical redo information will be written in terms of the log. So, we said that physical undo redo in terms of updates will be created because the old value information is required for roll back.

Now, when the operation completes you write the transaction again, the identity of the operation, you mark that operation as ended and you do add something interesting this is the logical undo information. So, this is actually an operation which can undo the operation that you have done. So, if you have done an insert, this will become a delete, corresponding delete operation. So, this is what is called logical logging, in contrast logging of old value new value information that you do here is the physical logging and the corresponding physical log records.

(Refer Slide Time: 14:25)

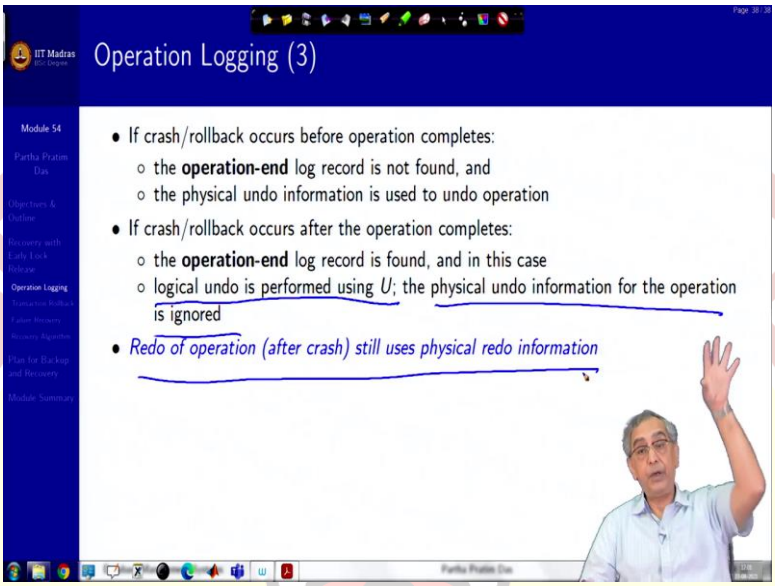


So, to understand a little bit more, a simple example. So, suppose you are doing an insert of a key, record-id pair; K5, RID7 into index I9, this is a situation and let us say this operation transaction is T1, the operation id is O1. So, you have T1, O1 operation begins certain tasks are done and you have specifically the physical redo steps for insert which is; X is the transaction's variable where the value of the key is computed which was 10 and is now being updated to K5. So, that is got changed to K5.

And another is Y, which was 45 being updated to the record-id RID7, these are physical records and then you want to with this insert that O1 is doing, we are seeing that you want to keep the track that you will be able to undo it. So, we say that transaction, the operation ID,

the operation end, but you write the logical undo operation, since you have done an insert of this key value in index 9, where I delete from index I9 key K5 record-id RID7. So, if you need to do that, if you need to roll back, you will be able to do this delete, get rid of it and then do the rest of the redo undo stuff.

(Refer Slide Time: 16:25)

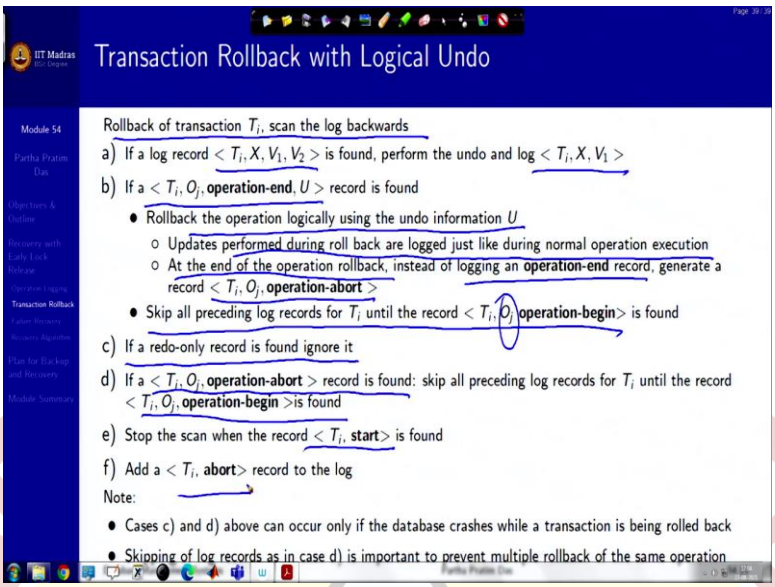


So, if the crash or rollback occurs before the operation completes, then the operation end log record will not be found and the physical undo information is used to do the undo operation which is normal. If the crash or rollback occurs after operation completes, so lock release have happened, changes have happened, then the operation end log record is found. And in this case, logical undo is performed using because operation is completed.

So, you have to do that logical undo using U the undo operation which was deleted in the last case. And once you have done that, you do not need to do this physical undo, so you just ignore that and then you redo operation, which will still use the physical redo operation. So, it is basically simplifying the whole thing to a great extent that if an early lock situation, if an operation has not completed, then the changes in that B plus tree has not happened or in that data structure has not happened.

So, all that you need to do is to physical undo of your variable values. If it has completed, then you need to effect the change in that data structure by doing the logical undo operation performed and you ignored the physical undo part. So, you are back to when the operation begin. And therefore, what you need to do is the physical redo part to get back to the point where you feint. So, this is your basic logging, logical logging mechanism.

(Refer Slide Time: 18:32)



So, now, to put transaction rollback with logical undo together if we put this whole thing together. So, rollback of transaction Ti will scan the log backwards, if you find such an entry, so, this is a physical change that is being shown. So, you will write the compensation, perform the undo and do the compensation log.

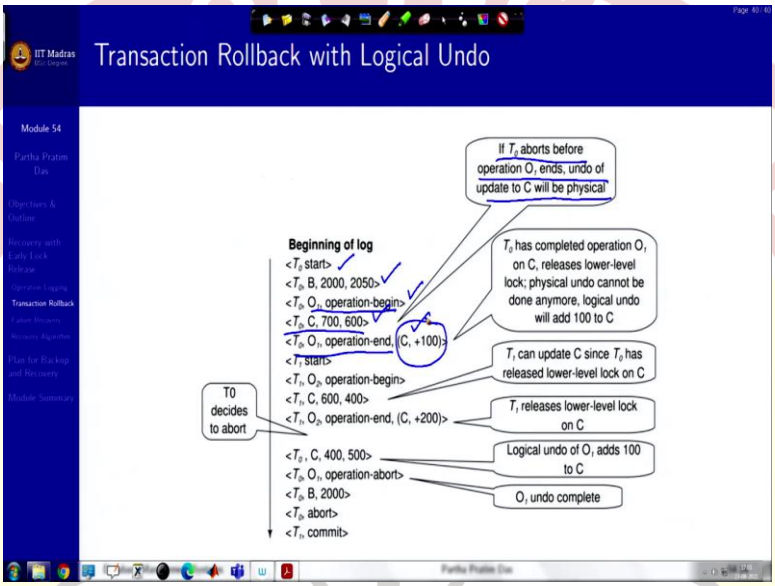
Going backwards, if you find an operation end with you, then rollback the operation logically using the undo operation, you are rolling back right, so you are going backwards. Update performed rollback, so log just like normal operation execution. Once you do this, that is basically doing another operation. So, you update during this for example, you had done insert which you are having to rollback by doing a delete for this delete, you do the logging exactly as in the normal operation.

Because you do not know, you may need to, you may have further failure, you may need to do further rollbacks. So, at the end of operation rollback instead of logging and operation end record, because this operation was not a successful one, this is the effect of rollback. So, you write operation abort exactly similar concept of as you wrote Ti abort.

And then you skip all proceeding records of Ti until you find the operation begin of the same operation, because you do not need to do anything after that. If a redo only record is found, ignore it, you do not need to take care of that. If going back in this way, if you found and find an operation abort record, then you will skip everything up to the operation begin because if you find an operation abort record that means that some operation earlier were logically undone. So, you do not need to do anything for that.

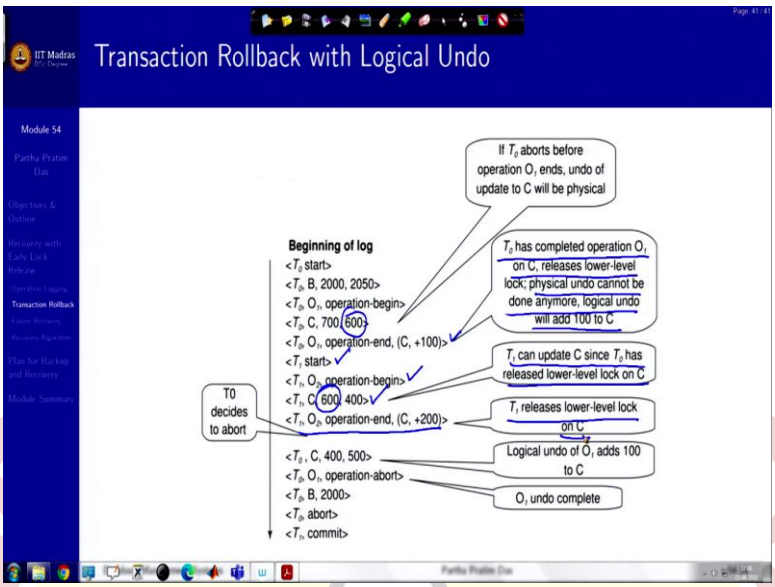
And finally, when you get to the record, when you get back to Ti start, the beginning of this transaction and naturally what you do you add Ti abort. So, this is how you will roll back the transaction with a logical undo. So, far, we are rolling back transactions only assuming that you are under the paradigm of two phase locking within the transaction and not changes happening in the data structure back into the disk, but now with these operations, logical operations, you will be able to take care of this in the transaction.

(Refer Slide Time: 21:49)



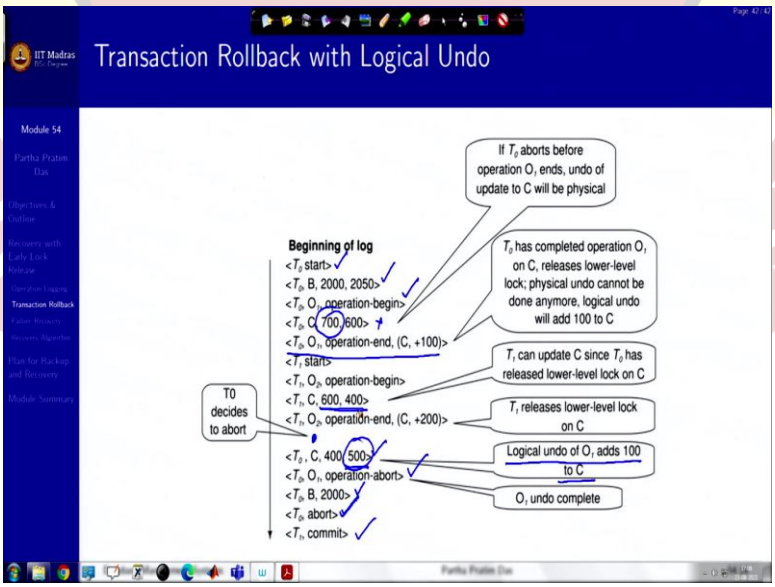
So, example. So, T0 starts, writes B operation begin you start to take that change there. Operation C is again another write from 700 to 600. So, if T0 aborts before operation, O1 ends this begin. Update C, update of C will be, update to C will be physical. So, for doing this you write operation end and you write undoing operation, why this is (C + 100)? Because, here this is just some shortcuts being done, yes, here you have changed C from 700 to 600. So, if you have to undo that logically, then you will have to add 100 to C, so, you write down that.

(Refer Slide Time: 22:57)



So, T1 has completed operation O1, release the low level locks are released, physical undo cannot be done anymore, logical undo will add 100, we are at this point. Then another transaction T1 starts and with another operation O2 which updates C from 600 to 400. And the operation end writes the corresponding logical undo operation which is adding 200 to it, 600 minus 400. So, T1 can update see since T0 has released the low level lock on C that is why it is possible T1 in comes to end up operation O2 releases a low level lock on C.

(Refer Slide Time: 23:56)

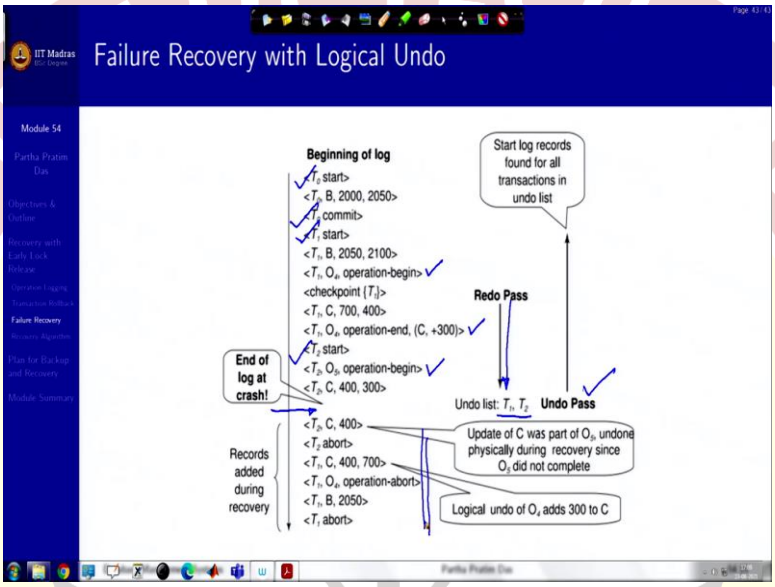


At this point, at this point both T0 is executing and T1 is executing at this point T0 decides to abort. If T0 decides to abort, you have to go backwards and find out when you get an operation end which you will get here. So, you have to do a logical undo to add 100 to it. So,

it becomes 500 and your output on your log a physical log record, you get to operation begin skipping this and you write operation abort. B had not been written in the data structure.

So, for B you do a physical undo and you write the compensation log record and then you come to the beginning of T0 you write T0 abort, T1 eventually commits and when T1 commits the value of C is 500 which was originally 700. And the effect of reducing it by 200 is surviving, the effect of reducing it by further 100 which T0 had done has been annulled.

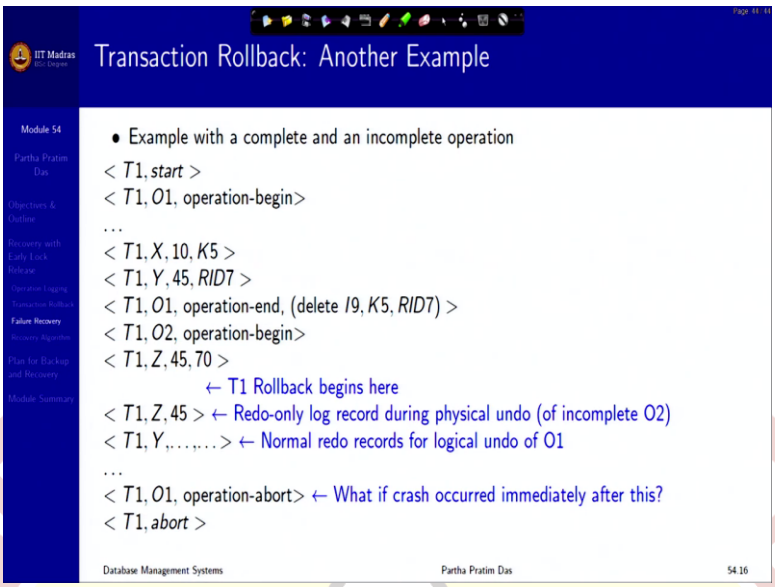
(Refer Slide Time: 25:31)



So, this is basically, Now, similarly, if you have say failure with logical, if you have a failure recovery, then this is the point where your log was when you actually have a crash, you have operation begin here, corresponding operation end here, you have operation begin here and has not been ended as yet. So, what you will have to do is to first do a redo, redo with what? T0 is started, T1 has started, T2 has started T0 has committed.

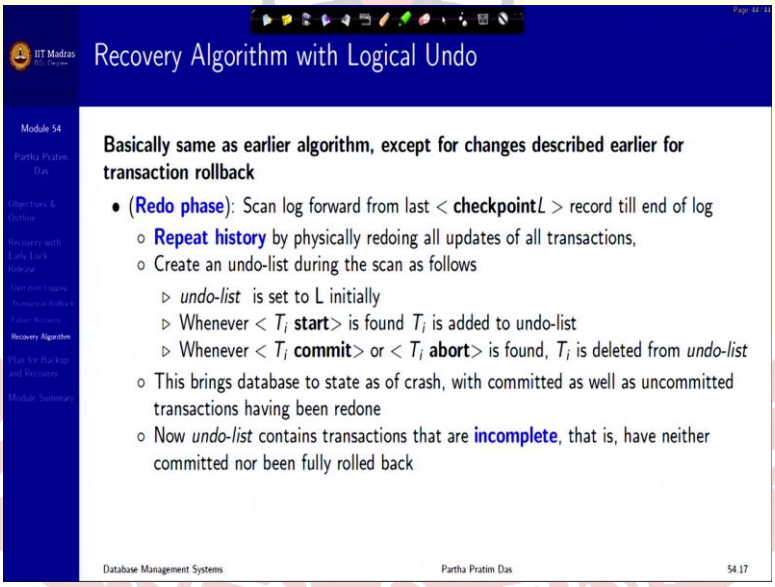
So, your redo has to create the undo list which is T1 T2, because T0 has already committed and on which you will do the undo it is very easy to see that these records will be created in the log, go through it step by step and convince yourself that this is indeed will achieve the actual consistent state of the database.

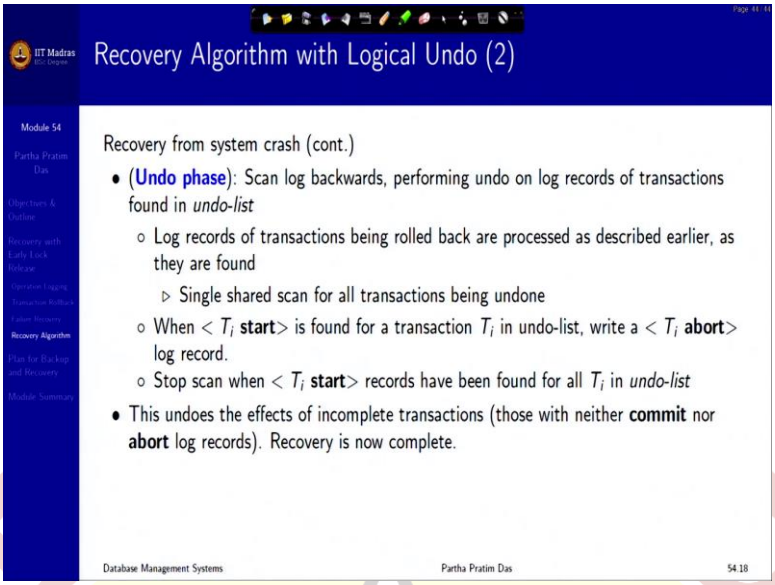
(Refer Slide Time: 26:46)



Here is another example given continuing on the earlier index change example for you to study and understand.

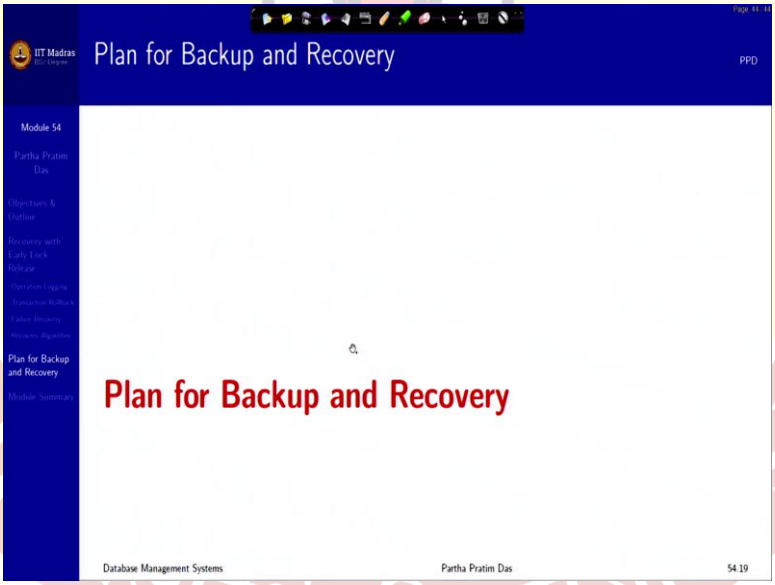
(Refer Slide Time: 26:55)

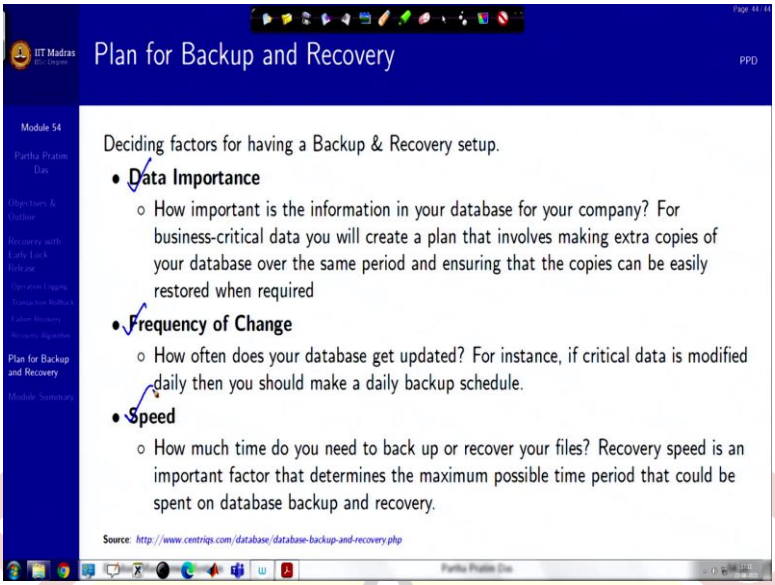




And finally, in the last two slides of this section, I have summarized what is the redo phase with logical undo and what is the recovery, undo phase of the recovery with logical undo. This is just the abstract summarization for your easy consumption.

(Refer Slide Time: 27:19)

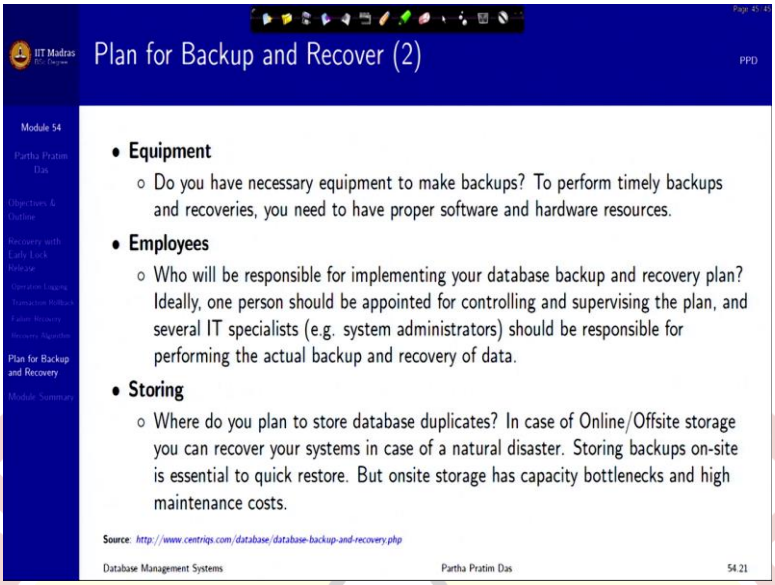




Now, coming to the plan for backup and recovery, how should you plan it? Anything I mean, we have talked about so many different options, so many different hardware that you might need and what is the frequency of backups that you will do and so on. So, these factors you should keep in mind for example, one is how important is the data, you may not again have the same strategy of backup for all your data, some data may be more critical than that.

Then, what is the frequency of change that you expect in the database, the critical data is modified daily. So, should come in the daily backup schedule like accounts data, who is your account holder, the personal data of that account holder and they are rarely changing, but the account transactions are frequently changing, so you will have to have different strategies. What is the speed of recovery that you need, can you announce a downtime to recover your files and so on? Because everything has a cost involved with it a business decision involved with it.

(Refer Slide Time: 28:33)

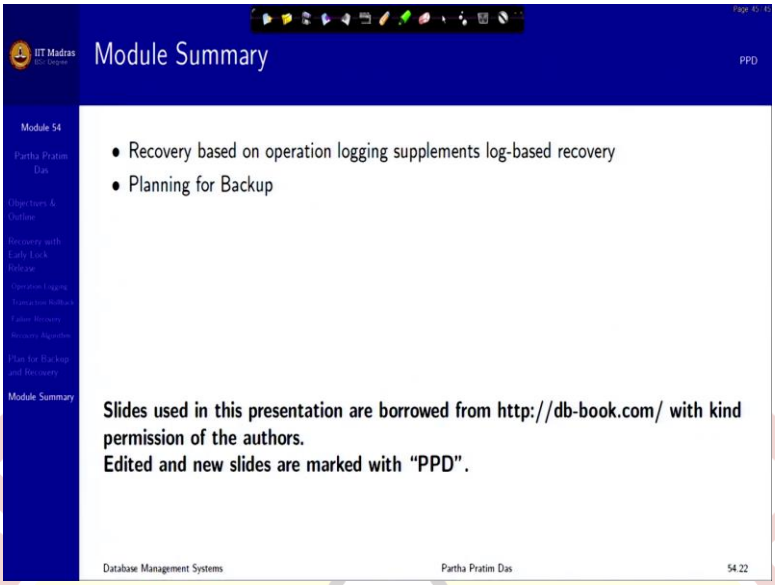


What kind of equipment that is hardware, software resources that we will need for the backups is what will decide much of the cost. Another big part of the cost is the staff, the employees, who will be responsible for implementing that, if you want to have a very agile recovery scheme working at a high speed then you will need a big team of skilled employees for doing that which maybe may not be cost effective.

Then where do you plan to store database duplicates? Would you keep them on site in another storage or offsite storage? Will you keep multiple copies and so on so forth? So, all of these factors will have to be taken into account to plan for the data backup, database backup and recovery.

Basic point is most often some seniors should be doing it, we just included the point so that as an initial database engineer, you should know that these costs are involved, these factors are involved. So, that when those decisions are made in the organization, you will be able to appreciate them better.

(Refer Slide Time: 29:48)



So, this brings us to the end of this module where we have discussed primarily the recovery based on early lock release which is reality and kind of covers the entire database backup and recovery paradigm in a certain level of detail. So, thank you very much for your attention. We will meet in the next module where I will talk about a very specific stable storage, that is very popular, called the RAID mechanism, which will be interesting for you to know and know the range of solutions you have for the stable storage. Thank you very much.